

Technical Requirement Document (TRD)

Time-Off Microservice - ExampleHR

Version: 1.0

Date: 2026-05-11

Stack: NestJS · SQLite · TypeScript

1. Problem Statement

ExampleHR employees submit time-off requests through ExampleHR, but the Human Capital Management system (HCM, e.g. Workday / SAP) is the authoritative **Source of Truth** for leave balances. Keeping these systems in sync is hard because:

- HCM balances change **outside** ExampleHR (e.g. work-anniversary bonuses, year-start refreshes).
 - HCM may or may not reliably return errors on overdraft - we must be **defensively correct**.
 - Concurrent requests from the same employee could together exceed their balance if not carefully serialised.
 - HCM exposes both a **real-time single-record API** and a **batch full-corpus endpoint**.
-

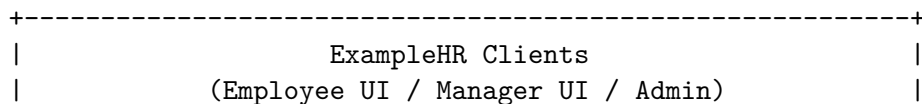
2. Goals

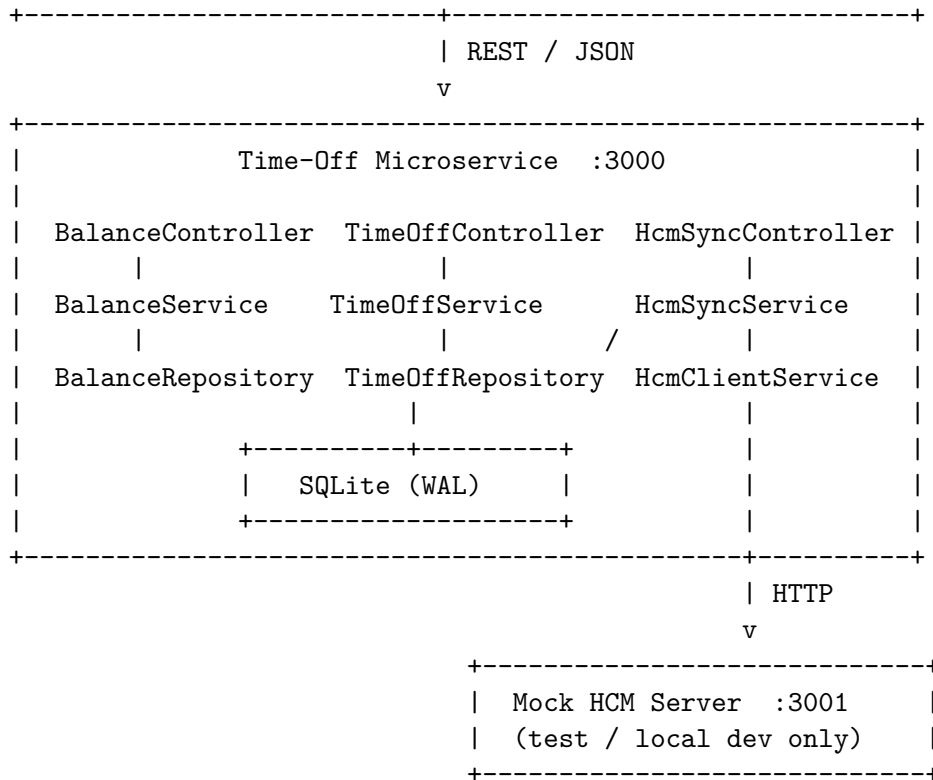
Goal	Success Criterion
Accurate balance display	Employee always sees <code>available = HCM_balance - locally_pending</code>
Balance integrity	It is impossible to over-allocate balance, even under concurrent load
HCM sync resilience	External HCM changes are reconciled without data loss
Defensive validation	Local balance check is the last line of defence even if HCM omits errors
Audit trail	Every balance mutation is recorded with source and old/new values
Test coverage	≥ 80 % line coverage; all concurrency and sync edge-cases are exercised

Non-Goals

- Full authentication/authorisation system (JWT stub is sufficient for the assignment).
 - Multi-tenancy (single-tenant assumed).
 - Leave policy rules (weekends, holidays) - **days** is caller-supplied.
 - Push notifications to employees.
-

3. Architecture Overview





Data flow summary: 1. Requests flow into TimeOffController -> TimeOffService. 2. Before accepting a request, the service checks local available balance **and** calls the HCM real-time API to get an authoritative current balance. 3. On manager approval, the service submits the request to HCM. 4. When HCM pushes a batch sync, HcmSyncService reconciles local shadow balances.

4. Data Model

-- Thin employee/location reference tables (populated by batch sync or seed)

```

CREATE TABLE employees (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  created_at  TEXT DEFAULT (datetime('now'))
);

```

```

CREATE TABLE locations (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  created_at  TEXT DEFAULT (datetime('now'))
);

```

-- Shadow copy of HCM balances, extended with local pending tracking

```

CREATE TABLE leave_balances (
  id          TEXT PRIMARY KEY,
  employee_id TEXT NOT NULL REFERENCES employees(id),

```

```

location_id      TEXT      NOT NULL REFERENCES locations(id),
leave_type       TEXT      NOT NULL DEFAULT 'VACATION',
hcm_balance      REAL      NOT NULL DEFAULT 0,      -- authoritative HCM available balance
pending_days     REAL      NOT NULL DEFAULT 0,      -- sum of locally PENDING requests
version          INTEGER   NOT NULL DEFAULT 1,      -- optimistic lock counter
hcm_last_synced_at TEXT,
created_at       TEXT      DEFAULT (datetime('now')),
updated_at       TEXT      DEFAULT (datetime('now')),
UNIQUE (employee_id, location_id, leave_type)
);

-- Derived view: available_days = hcm_balance - pending_days
-- NOT stored to avoid inconsistency; always computed on read.

CREATE TABLE time_off_requests (
  id              TEXT      PRIMARY KEY,
  employee_id     TEXT      NOT NULL REFERENCES employees(id),
  location_id     TEXT      NOT NULL REFERENCES locations(id),
  leave_type      TEXT      NOT NULL DEFAULT 'VACATION',
  start_date      TEXT      NOT NULL,              -- ISO-8601 date
  end_date        TEXT      NOT NULL,
  days            REAL      NOT NULL,
  status          TEXT      NOT NULL DEFAULT 'PENDING',
  -- PENDING / APPROVED / REJECTED / CANCELLED / COMPLETED
  notes          TEXT,
  hcm_reference_id TEXT,                          -- ID returned by HCM on submission
  hcm_submitted_at TEXT,
  idempotency_key TEXT      UNIQUE,
  created_at      TEXT      DEFAULT (datetime('now')),
  updated_at      TEXT      DEFAULT (datetime('now'))
);

-- Immutable audit log for every balance mutation
CREATE TABLE balance_audit_logs (
  id              TEXT      PRIMARY KEY,
  employee_id     TEXT      NOT NULL,
  location_id     TEXT      NOT NULL,
  leave_type      TEXT      NOT NULL,
  source          TEXT      NOT NULL,              -- 'REQUEST_CREATED' / 'REQUEST_CANCELLED' / 'REQUEST_APPROVED'
  -- / 'BATCH_SYNC' / 'REALTIME_SYNC' / 'MANUAL_CORRECTION'
  prev_hcm_balance REAL,
  new_hcm_balance REAL,
  prev_pending    REAL,
  new_pending     REAL,
  reference_id    TEXT,                          -- request ID or sync batch ID
  created_at      TEXT      DEFAULT (datetime('now'))
);

```

Invariant: `available_days = hcm_balance - pending_days >= 0` must hold at all times.

Invariant: `pending_days` = sum of days for all requests in {PENDING} state for that (employee, location, leave_type).

5. API Specification

All endpoints return:

```
{ "data": <payload | null>, "error": <string | null> }
```

5.1 Balance Endpoints

GET /api/v1/employees/:employeeId/balances Returns all balances for an employee.

Response 200:

```
{
  "data": [
    {
      "locationId": "LOC-1",
      "leaveType": "VACATION",
      "hcmBalance": 10.0,
      "pendingDays": 3.0,
      "availableDays": 7.0,
      "lastSyncedAt": "2026-05-10T09:00:00Z"
    }
  ]
}
```

GET /api/v1/employees/:employeeId/balances/:locationId Single balance. Accepts ?leaveType=VACATION (default: VACATION).

POST /api/v1/employees/:employeeId/balances/refresh Triggers a real-time HCM balance fetch and updates local shadow. Returns updated balance.

5.2 Time-Off Request Endpoints

POST /api/v1/time-off/requests Creates a time-off request. Idempotency-Key header supported.

Request:

```
{
  "employeeId": "EMP-1",
  "locationId": "LOC-1",
  "leaveType": "VACATION",
  "startDate": "2026-06-01",
  "endDate": "2026-06-02",
}
```

```

    "days": 2.0,
    "notes": "optional"
}

```

Response 201:

```

{
  "data": {
    "requestId": "uuid",
    "status": "PENDING",
    "availableAfterReservation": 5.0
  }
}

```

Errors: - 400 - validation failure (missing fields, endDate < startDate, days <= 0) - 409 - insufficient balance (includes available balance in error body) - 422 - HCM rejected the combination (invalid dimensions) - 429 - idempotency key collision with conflicting body

GET /api/v1/time-off/requests/:requestId

GET /api/v1/time-off/requests?employeeId=&locationId=&status=&page=&limit=

PATCH /api/v1/time-off/requests/:requestId/approve Manager action. Submits request to HCM and transitions status to APPROVED.

Response 200: updated request DTO.

Errors: 404, 409 (wrong state), 502 (HCM submission failed).

PATCH /api/v1/time-off/requests/:requestId/reject Manager action. Transitions to REJECTED, releases pending_days reservation.

Request body (optional): { "reason": "string" }

DELETE /api/v1/time-off/requests/:requestId Employee cancels request. If PENDING: release pending_days. If APPROVED: call HCM to cancel and credit back balance.

Errors: 409 (cannot cancel COMPLETED or REJECTED requests).

5.3 HCM Sync Endpoints (called by HCM or internal scheduler)

POST /api/v1/hcm/sync/batch Receives full corpus of balances from HCM. Authenticated with X-HCM-API-Key header.

Request:

```

{
  "batchId": "uuid",
  "syncedAt": "2026-05-11T08:00:00Z",
  "balances": [
    {

```

```

        "employeeId": "EMP-1",
        "locationId": "LOC-1",
        "leaveType": "VACATION",
        "availableBalance": 8.5
    }
]
}

```

Response 200: { "data": { "processed": 150, "discrepanciesFound": 2 } }

POST /api/v1/hcm/sync/single Single balance update webhook. Same auth as batch.

```

{
    "employeeId": "EMP-1",
    "locationId": "LOC-1",
    "leaveType": "VACATION",
    "availableBalance": 12.0,
    "reason": "ANNIVERSARY_BONUS"
}

```

5.4 Mock HCM Server API (:3001)

```

GET /hcm/balances/:employeeId/:locationId/:leaveType -> { availableBalance }
POST /hcm/balances/deduct { employeeId, locationId, leaveType, days } -> { newBalance } | 422
POST /hcm/balances/credit { employeeId, locationId, leaveType, days } -> { newBalance }
POST /hcm/sync/push-to-examplehr (triggers batch sync to localhost:3000)

# Debug / test control
PUT /hcm/debug/balance { employeeId, locationId, leaveType, balance }
POST /hcm/debug/anniversary { employeeId, locationId, leaveType, bonusDays }
POST /hcm/debug/reset
GET /hcm/debug/state

```

6. Core Algorithms

6.1 Create Time-Off Request

```

function createRequest(req):
    1. validate input (dates, days > 0)
    2. check idempotency key -> return cached response if exists
    3. BEGIN IMMEDIATE TRANSACTION
        a. SELECT balance WHERE (employee, location, leaveType) FOR UPDATE (SQLite: IMMEDIATE lock)
        b. available = balance.hcm_balance - balance.pending_days
        c. IF available < req.days -> ROLLBACK -> return 409 (fast local check)
    4. CALL HCM real-time API: GET /hcm/balances/{emp}/{loc}/{type}
        a. hcm_available = response.availableBalance
        b. authoritative_available = hcm_available - balance.pending_days

```

```

    c. IF authoritative_available < req.days -> ROLLBACK -> return 409
       (HCM disagrees with our local shadow - update shadow before returning)
5. WITHIN TRANSACTION:
   UPDATE leave_balances
     SET pending_days = pending_days + req.days,
         version = version + 1,
         updated_at = now()
     WHERE id = balance.id AND version = balance.version  <- optimistic lock
   IF rowsAffected == 0 -> ROLLBACK -> retry (max 3 times) or return 409
   INSERT time_off_request (status=PENDING)
   INSERT balance_audit_log (source=REQUEST_CREATED)
6. COMMIT
7. return 201

```

Why real-time HCM call at step 4? The local shadow may be stale (e.g. HCM just applied an anniversary bonus or a manager cancelled something directly in HCM). The real-time call is the authoritative check.

Why optimistic lock at step 5? Two concurrent requests may both pass step 3 and step 4. The version check ensures only one wins the UPDATE; the other retries and now sees reduced available balance.

6.2 Approve Request

```

function approveRequest(requestId):
1. Load request; assert status == PENDING
2. Load balance for (employee, location, leaveType)
3. CALL HCM POST /hcm/balances/deduct { days: req.days }
   a. IF HCM returns 422 (insufficient balance in HCM):
      -> log warning, return 409 to manager with explanation
      -> optionally trigger a balance refresh (the shadow is stale)
   b. IF HCM returns 5xx or times out:
      -> return 502 to manager ("HCM unavailable, retry")
      -> do NOT transition state (request remains PENDING)
4. BEGIN IMMEDIATE TRANSACTION
   UPDATE time_off_requests SET status=APPROVED, hcm_reference_id=..., hcm_submitted_at=now()
   UPDATE leave_balances
     SET hcm_balance = hcm_balance - req.days,
         pending_days = pending_days - req.days,
         -- net effect on available: zero (both sides shrink equally)
         version = version + 1
     WHERE id = balance.id AND version = balance.version
   INSERT balance_audit_log (source=REQUEST_APPROVED)
5. COMMIT
6. return 200

```

Why does available stay unchanged on approval? When the request was created, pending_days was incremented by days. On approval, hcm_balance decreases by days AND

pending_days decreases by days. The net available = hcm_balance - pending_days is unchanged.

6.3 Batch Sync Reconciliation

```
function processBatchSync(batchPayload):
  discrepancies = []
  FOR EACH balance IN batchPayload.balances:
    1. Upsert (employee, location, leaveType) in leave_balances:
      new_hcm_balance = payload.availableBalance
    2. Load sum of PENDING request days -> pending_from_db
    3. new_available = new_hcm_balance - pending_from_db
    4. IF new_available < 0:
      discrepancies.push({ employee, location, diff: new_available })
      -- HCM has less balance than our pending requests imply
      -- this is a WARNING; do not automatically cancel requests
    5. UPDATE leave_balances
      SET hcm_balance = new_hcm_balance,
          hcm_last_synced_at = batchPayload.syncedAt,
          version = version + 1
      WHERE ...
    6. INSERT balance_audit_log (source=BATCH_SYNC)
  return { processed: N, discrepanciesFound: discrepancies.length }
```

Discrepancy handling: A discrepancy (negative available) means HCM shows fewer days than our pending requests would consume. This is logged and surfaced in the response but does **not** auto-cancel requests - a human/admin should resolve it.

6.4 Cancel Approved Request

```
function cancelRequest(requestId):
  1. Load request; assert status IN [PENDING, APPROVED]
  2. IF status == PENDING:
    BEGIN TRANSACTION
    UPDATE request SET status=CANCELLED
    UPDATE leave_balances SET pending_days = pending_days - req.days
    INSERT audit log
    COMMIT -> return 200
  3. IF status == APPROVED:
    CALL HCM POST /hcm/balances/credit { days: req.days }
    IF HCM fails -> return 502, do not change state
    BEGIN TRANSACTION
    UPDATE request SET status=CANCELLED
    UPDATE leave_balances SET hcm_balance = hcm_balance + req.days
    INSERT audit log
    COMMIT -> return 200
```

7. Challenges & Proposed Solutions

Challenge 1 - Concurrent Over-Allocation

Problem: Two concurrent requests may both pass balance checks and together exceed available balance.

Solution: SQLite BEGIN IMMEDIATE transaction + optimistic version-column locking. Only one writer at a time can commit; the loser retries with fresh data.

Challenge 2 - Stale Local Shadow Balance

Problem: HCM may change a balance (anniversary bonus, manual correction) without notifying ExampleHR.

Solution:

- Real-time HCM API call during request creation (authoritative pre-flight).
- HCM batch sync endpoint (POST /hcm/sync/batch) for periodic full reconciliation.
- HCM single-balance webhook (POST /hcm/sync/single) for immediate push notifications.
- Employees can trigger manual refresh (POST /employees/:id/balances/refresh).

Challenge 3 - HCM Unreliable Error Reporting

Problem: HCM may not always reject overdraft requests.

Solution: Local balance tracking is the primary guard. The real-time HCM check is secondary. Even if HCM accepts a bad request, our local available guard rejects it first (step 3c in the create flow).

Challenge 4 - HCM Submission Failure on Approval

Problem: Manager approves, but HCM API call fails (network partition, HCM downtime).

Solution: Return 502 to manager - request stays PENDING. Manager retries approval when HCM is back. No state corruption occurs because approval is atomic: either both the DB write and HCM call succeed, or neither is committed.

Future improvement: Outbox pattern - write HCM submission intent to a DB table; a background worker retries it.

Challenge 5 - Batch Sync Race with Active Requests

Problem: A batch sync arrives while a request is being created concurrently.

Solution: Both operations use BEGIN IMMEDIATE on the leave_balances row. The sync is a regular DB write protected by the same version lock as request creation.

8. Alternatives Considered

A - Always submit to HCM on request creation (not on approval)

Pros: HCM is always aware of all requests immediately.

Rejected because: Manager may reject the request; we would need to cancel in HCM for every

rejected request, adding HCM API calls and complexity. Also, HCM business rules may differ from ExampleHR approval rules.

B - No local balance shadow; always call HCM real-time

Pros: Always authoritative.

Rejected because: HCM latency adds to every balance read (bad UX), HCM downtime makes ExampleHR unreadable, and there's no way to show pending impact to the employee without local tracking.

C - Pessimistic DB locking instead of optimistic

Pros: Simpler concurrency model.

Rejected because: SQLite WAL mode performs better under read-heavy load with optimistic locking. Pessimistic locks would serialize all reads on the balance row, not just writes.

D - PostgreSQL instead of SQLite

Pros: Better horizontal scaling, native advisory locks, more robust concurrent writes.

Context: Assignment specifies SQLite. However, the repository pattern abstracts the DB layer so migration to PostgreSQL in production is straightforward - only the TypeORM driver and WAL configuration differ.

9. Non-Functional Requirements

Correctness & Data Integrity

The highest-priority NFR for a balance-management system.

- **Optimistic locking on every balance write.** The `leave_balances.version` column is incremented on every UPDATE. All writes use `UPDATE ... WHERE id = ? AND version = ?expected`; zero rows affected throws `OptimisticLockError` and retries up to 5 times with jittered backoff. No write can silently corrupt a balance.
- **BEGIN IMMEDIATE transactions.** All balance mutations acquire the SQLite write lock upfront, preventing two concurrent transactions from both passing the available-balance check and together exceeding the limit. TypeORM's `QueryRunner` is used with raw `BEGIN IMMEDIATE` rather than the default `BEGIN DEFERRED`.
- **Two-guard balance check on request creation.** The local shadow check (fast, catches the obvious case) runs first. A live HCM real-time call then runs as the authoritative check - catching cases where the HCM balance decreased externally but our shadow hasn't caught up. Even if HCM silently accepts an overdraft, the local guard blocks it.
- **Audit log written in the same transaction.** Every balance mutation (creation, approval, rejection, cancellation, sync) inserts a row into `balance_audit_logs` inside the same `BEGIN IMMEDIATE` block. There is no window where a balance changed without a log entry.
- **Audit log is append-only.** No UPDATE or DELETE operations on `balance_audit_logs` exist anywhere in the codebase. The table is a full, immutable replay log.
- **available_days is never stored.** It is always computed as `hcm_balance - pending_days` on read, eliminating an entire class of stale-computed-value bugs.

Reliability & Fault Tolerance

- **HCM unavailability is gracefully degraded.** If `getBalance` fails during request creation, the service falls back to the local shadow balance and logs a warning - employees can still submit requests. If `deductBalance` fails during approval, a 502 is returned and the database is left unchanged; the request stays `PENDING` and the manager can retry.
- **HCM client retries with exponential backoff.** `axios-retry` is configured for 3 attempts at 100ms / 400ms / 1600ms on 5xx responses and network timeouts only. 4xx responses (including 422 insufficient balance) are not retried - they are definitive HCM signals.
- **Optimistic-lock retry loop.** The entire transaction body (including re-reading the balance row) is wrapped in `retryOnOptimisticLock`, which retries up to 5 times on `OptimisticLockError` or SQLite “database is locked” errors. Each retry sees fresh data.
- **PRAGMA busy_timeout = 5000.** SQLite waits up to 5 seconds for a write lock before returning a locked error, giving the retry loop time to succeed under burst load.
- **Transactional atomicity.** Every multi-step operation (insert request + update balance + write audit log) is wrapped in `BEGIN IMMEDIATE` - either all steps commit or none do.
- **Known gap - outbox pattern.** If `deductBalance` succeeds but the subsequent DB write fails after all retries (extremely rare), the HCM has deducted the balance but `ExampleHR` has no record of it. A `CRITICAL` log entry with the HCM reference ID is emitted, but there is no automated reconciliation. An outbox table with a background worker would close this gap in production.

Scalability

- **SQLite WAL mode (PRAGMA journal_mode=WAL).** Concurrent readers are allowed even during an active write. Read-heavy workloads (employees checking balances) are not blocked by pending writes.
- **Stateless service.** No in-process session state or connection-local caches. Multiple instances can run behind a load balancer - the only constraint is the single-writer SQLite file. Migration to PostgreSQL removes this constraint.
- **Repository pattern abstracts the DB layer.** Swapping `better-sqlite3` for `pg` requires changes only in `DatabaseModule` and `data-source.ts` - no service or repository code changes.
- **Paginated list endpoints.** All list queries enforce `page` + `limit` with a maximum of 100 results per page. No unbounded full-table scans are exposed via the API.
- **Per-entry batch sync transactions.** Each balance entry in a batch sync payload is processed in its own isolated `BEGIN IMMEDIATE` transaction. A 10,000-row batch does not create a single monolithic lock that blocks all other writes for seconds.
- **Known ceiling.** SQLite serialises all writers. Under high concurrent write load (thousands of simultaneous requests), `BEGIN IMMEDIATE` contention will saturate. This is an intentional scope trade-off for the assignment; the architecture is designed to make the PostgreSQL migration a configuration change.

Security

- **Timing-safe API key comparison.** `crypto.timingSafeEqual` is used when validating the X-HCM-API-Key header in `HcmApiKeyGuard`, preventing timing side-channel attacks.
 - **JWT guard with correct interface.** `JwtAuthGuard` validates the `Authorization: Bearer` header. The current implementation is a stub (any non-empty token accepted); the guard interface is correct and ready for real JWT verification without touching service code.
 - **Strict DTO validation.** All DTOs use `class-validator` with `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })`. Unknown fields are stripped and rejected, preventing mass-assignment attacks.
 - **Fail-fast env validation.** `ConfigModule` validates all required environment variables at startup using `class-validator`. The service refuses to boot with missing or malformed configuration - no silent runtime fallbacks to insecure defaults.
 - **Rate limiting.** `@nestjs/throttler` enforces 100 requests/minute per IP across all endpoints.
 - **Secrets in environment only.** No secrets (API keys, JWT secret, DB path) are hardcoded. All come from environment variables.
 - **Internals never leak in error responses.** The global `HttpExceptionFilter` returns generic messages for 500 errors. Stack traces, internal state, and balance values for other employees never appear in responses.
-

Observability

- **Request ID propagation.** Every request generates a UUID via `LoggingInterceptor`, attaches it to the `X-Request-Id` response header, and includes it in all log entries for that request. Tracing a complete request through logs requires only that ID.
 - **Business-level audit trail.** `balance_audit_logs` is a queryable, append-only record of every balance change with source, previous values, new values, and reference ID. It is more useful for operations and compliance than application logs alone.
 - **Discrepancy detection.** Batch sync explicitly detects negative-available scenarios (HCM shows fewer days than pending requests consume) and logs a structured `WARN` entry with employee ID, location, leave type, and the exact diff.
 - **Interactive API documentation.** Swagger UI at `/api/docs` provides a fully annotated, always-up-to-date API surface. Both auth schemes (Bearer token and X-HCM-API-Key) are wired into the Swagger “Authorize” dialog. OpenAPI JSON is available at `/api/docs-json`.
-

Maintainability

- **Single HCM client boundary.** `HcmClientService` is the only class that makes outbound HTTP calls to HCM. All tests mock exactly this one boundary - no test needs to understand the axios configuration.
- **Single responsibility per layer.** `BalanceRepository` owns all optimistic-lock mutations. `TimeOffService` owns the state machine. `HcmSyncService` owns reconciliation logic. No cross-cutting concerns bleed between modules.
- **92% test coverage across 105 tests.** Unit, integration, and E2E suites provide a regression net that makes refactoring safe. All 5 mandatory regression scenarios (concurrent over-allocation, anniversary bonus, HCM 422 on approval, cancel approved, idempotency replay)

have dedicated named test files.

10. Testing Strategy

Unit Tests (Jest)

Test each service method in isolation with mocked repositories and a mocked `HcmClientService`.

Key scenarios: | Scenario | Test File | |-----|-----| | Create request - happy path | `time-off.service.spec.ts` | | Create request - insufficient local balance | `time-off.service.spec.ts` | | Create request - HCM real-time disagrees with local | `time-off.service.spec.ts` | | Create request - idempotency key deduplication | `time-off.service.spec.ts` | | Approve - HCM returns 422 (balance mismatch) | `time-off.service.spec.ts` | | Approve - HCM times out (stays PENDING) | `time-off.service.spec.ts` | | Cancel PENDING - pending_days released | `time-off.service.spec.ts` | | Cancel APPROVED - HCM credit called | `time-off.service.spec.ts` | | Batch sync - balance increase (anniversary) | `hcm-sync.service.spec.ts` | | Batch sync - balance decrease, negative available | `hcm-sync.service.spec.ts` | | Batch sync - new employee/location upserted | `hcm-sync.service.spec.ts` | | Optimistic lock retry on concurrent update | `balance.repository.spec.ts` |

Integration Tests (Jest + Supertest)

Spin up the full NestJS app against an in-memory SQLite database. Mock the `HcmClientService` with `nock` or Jest module mocking.

Key scenarios: | Scenario | Test File | |-----|-----| | Full lifecycle: create -> approve -> cancel | `time-off.e2e-spec.ts` | | Concurrent requests - only one succeeds | `concurrency.e2e-spec.ts` | | Batch sync updates available balance | `hcm-sync.e2e-spec.ts` | | Reject, then employee can request again | `time-off.e2e-spec.ts` | | Pagination of requests | `time-off.e2e-spec.ts` |

E2E Tests with Live Mock HCM

Start both services (main on :3000, mock HCM on :3001) in `beforeAll`. Use the mock HCM debug API to set up state, then drive the flow through ExampleHR APIs.

Key scenarios: | Scenario | Test File | |-----|-----| | Anniversary bonus -> balance auto-corrects via batch sync | `anniversary-bonus.e2e-spec.ts` | | ExampleHR request, HCM balance changes independently mid-flight | `hcm-divergence.e2e-spec.ts` | | HCM returns 422 during approval | `hcm-rejection.e2e-spec.ts` | | Two simultaneous POSTs that together exceed balance | `race-condition.e2e-spec.ts` |

Coverage Target

`jest --coverage` with threshold: lines: 80, branches: 75, functions: 80.

11. Mock HCM Design

The mock HCM is a **separate NestJS application** in `mock-hcm/` that:

- Maintains in-memory balance state per (`employeeId`, `locationId`, `leaveType`).
- Implements HCM deduct logic: returns 422 when `availableBalance < requestedDays`.
- Exposes a debug API for test control (set balance, trigger batch push, simulate anniversary).
- On `POST /hcm/debug/anniversary`, increments the specified balance and immediately calls `POST http://localhost:3000/api/v1/hcm/sync/single` to push the update.
- On `POST /hcm/sync/push-to-examplehr`, packages all current state into a batch sync payload and calls `POST http://localhost:3000/api/v1/hcm/sync/batch`.

The mock is intentionally minimal - no persistence, restarts clean. Tests use `POST /hcm/debug/reset` between test suites.

12. Project Structure

```
/
+-- src/
|   +-- main.ts
|   +-- app.module.ts
|   +-- config/
|       +-- configuration.ts           # env vars with validation
|   +-- common/
|       +-- filters/http-exception.filter.ts
|       +-- guards/
|           +-- jwt-auth.guard.ts
|           +-- hcm-api-key.guard.ts
|       +-- interceptors/
|           +-- logging.interceptor.ts
|           +-- idempotency.interceptor.ts
|       +-- pipes/validation.pipe.ts
|   +-- database/
|       +-- database.module.ts
|       +-- database.service.ts       # TypeORM SQLite setup, WAL pragma
|       +-- migrations/               # numbered SQL migrations
|   +-- modules/
|       +-- balance/
|           +-- balance.module.ts
|           +-- balance.controller.ts
|           +-- balance.service.ts
|           +-- balance.repository.ts
|           +-- dto/balance-response.dto.ts
|           +-- entities/leave-balance.entity.ts
|       +-- time-off/
|           +-- time-off.module.ts
|           +-- time-off.controller.ts
|           +-- time-off.service.ts
|           +-- time-off.repository.ts
|           +-- dto/
```

```

|         | |   +-- create-time-off-request.dto.ts
|         | |   +-- time-off-response.dto.ts
|         |   +-- entities/time-off-request.entity.ts
|       +-- hcm-sync/
|         +-- hcm-sync.module.ts
|         +-- hcm-sync.controller.ts   # receives batch/webhook from HCM
|         +-- hcm-sync.service.ts     # reconciliation logic
|         +-- hcm-client.service.ts   # calls HCM API (injectable, mockable)
|         +-- dto/
|           +-- batch-sync.dto.ts
|           +-- single-sync.dto.ts
+-- mock-hcm/
|   +-- src/
|     |   +-- main.ts                 # listens on :3001
|     |   +-- app.module.ts
|     |   +-- hcm.controller.ts
|     |   +-- hcm.service.ts
|   +-- package.json
+-- test/
|   +-- unit/                         # *.spec.ts - Jest, fully mocked
|   +-- integration/                  # *.e2e-spec.ts - Supertest +nock
|   +-- e2e/                          # *.e2e-spec.ts - live mock HCM
+-- .env.example
+-- jest.config.ts
+-- package.json

```

13. Key Environment Variables

```

PORT=3000
DATABASE_PATH=./data/timeoff.sqlite
HCM_BASE_URL=http://localhost:3001
HCM_API_KEY=supersecret
HCM_REQUEST_TIMEOUT_MS=5000
HCM_RETRY_ATTEMPTS=3
JWT_SECRET=dev-secret
EXAMPLEHR_BASE_URL=http://localhost:3000      # used by mock HCM to push syncs

```